How often do you hear “but it works on my machine” when talking to developers? It is not easy to create an environment providing the libraries, frameworks, and runtime environments required by an application. Since 2013, Docker has made the concept of containers popular. As in logistics, a container in software development is a standardized unit that can be easily moved and delivered. In our experience, this method simplifies the development process significantly, especially when aiming for continuous deployment, which means shipping every change to test or production systems automatically.

In theory, you spin up a container based on the same image on your local machine, an on-premises server, and in the cloud. Boundaries exist only between UNIX/Linux and Windows, as well as Intel/AMD and ARM processors. In contrast, it is much more complicated to launch an Amazon Machine Image (AMI) on your local machine.

Containers also increase portability. In our opinion, it is much easier to move a containerized workload from on-premises to the cloud or to another cloud provider. But beware of the marketing promises by many vendors: it is still a lot of work to integrate your system with the target infrastructure.

We have guided several organizations in the adoption of containers. In doing so, we have observed that containers promote an important competency: building and running immutable servers. An immutable server is a server that you do not change once it is launched from an image. But what if you need to roll out a change? Create a new image and replace the old servers with servers launched from the new image. In theory, you could do the same thing with EC2 instances as well, and we highly recommend you do so. But because you are typically not able to log in to a running container to make